

Aether — CI/CD Integration

Pipeline-Native Deployment Automation

Aether is built for CI/CD from day one. Machine-readable output, dry-run previews, non-interactive mode, and policy gates make it a first-class citizen in any pipeline.

GitHub Actions • GitLab CI • JSON Output • Dry-Run • Policy Gates

GitHub Actions Workflow

A complete CI/CD pipeline that validates, builds, deploys, and optionally migrates your workload across runtimes.

.github/workflows/deploy.yml

```
name: Aether Deploy
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  AETHER_NAMESPACE: ${GITHUB_REF_NAME == 'main' && 'production' || 'staging' }

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Aether
        run: |
          curl -fsSL https://aether.dev/install.sh | sh
          aether --version

      - name: Validate Spec
        run: aether validate workload.yaml --output json --yes

      - name: Build Image
        run: aether build workload.yaml --yes

      - name: Dry-Run Deploy (PR only)
        if: github.event_name == 'pull_request'
        run: aether run workload.yaml --dry-run --yes --output json

      - name: Deploy (main only)
        if: github.ref == 'refs/heads/main'
        run: aether run workload.yaml --yes --output json

      - name: Verify Deployment
        if: github.ref == 'refs/heads/main'
        run: |
          aether status my-app --output json | jq '.state'
          # Fail if not running
          aether status my-app --output json | \
            jq -e '.state == "Running"'

      - name: Migrate (optional)
        if: github.ref == 'refs/heads/main' && inputs.migrate
        run: |
          aether migrate my-app kubernetes \
            --strategy blue-green --yes
```

validate



build



dry-run



deploy



verify



migrate

GitLab CI Configuration

Aether integrates into GitLab CI with the same flags and output formats.

.gitlab-ci.yml

```
stages:
  - validate
  - build
  - deploy
  - verify

variables:
  AETHER_NAMESPACE: "${CI_ENVIRONMENT_NAME}"

validate:
  stage: validate
  script:
    - aether validate workload.yaml --yes --output json
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"

build:
  stage: build
  script:
    - aether build workload.yaml --yes
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

deploy:preview:
  stage: deploy
  script:
    - aether run workload.yaml --dry-run --yes --output json
  environment:
    name: staging
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

deploy:production:
  stage: deploy
  script:
    - aether run workload.yaml --yes --output json
  environment:
    name: production
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

verify:
  stage: verify
  script:
    - aether status my-app --output json | jq -e '.state == "Running"'
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

The same CI flags work in every CI system: `--yes` for non-interactive mode, `--output json` for machine-readable output, and `--dry-run` for safe previews.

Machine-Readable Output

Every Aether command supports four output formats. JSON and YAML are designed for scripting and pipeline integration.

JSON Output (--output json)

```
$ aether status my-app --output json
{
  "name": "my-app",
  "runtime": "kubernetes",
  "state": "Running",
  "ready": true,
  "restart_count": 0,
  "namespace": "production",
  "age": "2h 15m"
}

# Pipe to jq for field extraction
$ aether status my-app --output json \
  | jq -r '.state'
Running

# Use --json shorthand (legacy)
$ aether status my-app --json
```

YAML Output (--output yaml)

```
$ aether status my-app --output yaml
name: my-app
runtime: kubernetes
state: Running
ready: true
restart_count: 0
namespace: production
age: 2h 15m

# Useful for config pipelines
$ aether status my-app --output yaml \
  >> deployment-report.yaml

# Wide format for extra detail
$ aether status --output wide
NAME      RUNTIME  STATE    READY   ...  IMAGE              PORTS
my-app    kube     Running  true    ...  my-app:latest      8080:80
my-web    podman   Running  true    ...  nginx:alpine        8081:80
```

CI-Friendly Flags Reference

Flag	Short	Description	CI Use Case
--output	-o	Output format: table, json, yaml, wide	Parse deploy results in scripts
--json		Shorthand for --output json	Legacy compatibility
--yes	-y	Skip all confirmation prompts	Non-interactive pipelines
--dry-run		Preview without executing	PR validation checks
--quiet	-q	Suppress all output except errors	Clean pipeline logs
--verbose	-v	Enable verbose logging	Debug failing pipelines

Dry-Run & Policy Gates

Preview deployments safely and enforce organizational policies in your pipeline.



Dry-Run Mode

The `--dry-run` flag shows exactly what Aether would create without applying any changes. Safe for PR checks and pipeline previews.

```
$ aether run workload.yaml --dry-run --yes
Would deploy my-web-app to kubernetes
  Namespace: production
  Image: my-app:latest
  Replicas: 2
  Ports: 8080:80
  Resources: 500m CPU, 256Mi RAM
  Service: ClusterIP
  Ingress: app.example.com (TLS)
  HPA: 2-10 replicas (70% CPU)

No changes applied.
```



Policy Gate Enforcement

Aether validates workloads against organizational policies before deployment. In CI, policy failures block the pipeline with a non-zero exit code.

```
$ aether run workload.yaml --yes
Error: Policy violation detected

FAIL resource-limits
  CPU limit exceeds maximum (4 > 2)
  Memory limit exceeds maximum (16Gi > 8Gi)

FAIL image-registry
  Image must use approved registry
  Got: docker.io/my-app:latest
  Expected: registry.example.com/*

Exit code: 1

# Skip policy checks (use with caution)
$ aether run workload.yaml \
  --skip-policy --yes
WARNING: Skipping policy checks
Deploying my-app to kubernetes...
```

Pipeline Best Practice

Use `--dry-run` in pull request pipelines for safe validation. Reserve `--skip-policy` for emergency deployments only — it should require explicit approval in your CI system. The `--yes` flag is mandatory in CI to prevent pipelines from hanging on confirmation prompts.

--dry-run

Safe Preview

--yes

Non-Interactive

--skip-policy

Emergency Override

Automate Everything

Aether fits into any CI/CD pipeline with zero custom scripting. Every command is idempotent, every output is parseable.

Minimal Pipeline Example

```
# Four commands for a complete CI/CD pipeline
aether validate workload.yaml --yes      # Lint + schema check
aether build workload.yaml --yes        # Build container image
aether run workload.yaml --yes --output json  # Deploy to runtime
aether status my-app --output json \
  | jq -e '.state == "Running"'          # Verify health

# Zero exit code = pipeline passes
# Non-zero exit code = pipeline fails
```

4

Output Formats

6

CI-Friendly Flags

0

Custom Scripts Needed

CI/CD Without the Glue Code

Aether replaces custom deployment scripts with a single binary that validates, builds, deploys, and verifies across Podman, Kubernetes, KubeVirt, and Metal3. Machine-readable output and non-zero exit codes make it a natural fit for any pipeline.